

# New upper bounds on covering codes $K_q(n, R)$ for alphabets of size six and seven

Mark Marosi\*

August 2026

## Abstract

We present improved upper bounds for eight entries of the standard tables of bounds on  $K_q(n, R)$ , the minimum cardinality of a  $q$ -ary code of length  $n$  with covering radius  $R$ , for  $q \in \{6, 7\}$ :  $K_6(7, 3) \leq 235$ ,  $K_6(8, 3) \leq 1054$ ,  $K_6(8, 4) \leq 169$ ,  $K_6(9, 4) \leq 719$ ,  $K_6(9, 5) \leq 126$ ,  $K_6(10, 4) \leq 2951$ ,  $K_6(10, 5) \leq 610$ , and  $K_7(8, 4) \leq 333$ . The previous best bounds, recorded in Kéri's tables (last updated 2011), all arose from general constructions (direct sums and related product rules) rather than from explicit search; to our knowledge these are the first improvements to any upper bound on  $K_q(n, R)$  with  $q \geq 5$  since 2011. The new bounds were found by focused local search seeded with the construction-based incumbents. All eight codes are given explicitly in the ancillary files, together with a standalone verifier; each code was checked by four independent exhaustive verification methods.

## 1 Introduction

Let  $\mathbb{Z}_q^n$  denote the Hamming space of  $q$ -ary words of length  $n$ . A code  $C \subseteq \mathbb{Z}_q^n$  has *covering radius*  $R$  if every word of  $\mathbb{Z}_q^n$  is within Hamming distance  $R$  of some codeword. The function

$$K_q(n, R) = \min\{|C| : C \subseteq \mathbb{Z}_q^n \text{ has covering radius } \leq R\}$$

is a central quantity of covering-code theory [1]; the binary case  $K_2(n, 1)$  includes the domination number of the hypercube, and the ternary case  $K_3(n, 1)$  is the classical football pool problem.

Upper bounds on  $K_q(n, R)$  are established by explicit constructions, and the state of the art is recorded in the tables of Kéri [2], which subsume and extend the earlier tables of Cohen, Honkala, Litsyn and Lobstein [1]. The last update to Kéri's tables was made in November 2011. Recent activity on  $K_q(n, R)$  has concentrated on *lower* bounds: Wu and Chen [4] improved binary lower bounds via congruence arguments, and Gijswijt and Polak [3] obtained many new lower bounds for  $q \leq 5$  from semidefinite programming. On the upper-bound side, for alphabets  $q \geq 5$  we are not aware of any published improvement since the 2011 freeze; a systematic search of the literature (arXiv, DBLP, OpenAlex, Lobstein's covering-code bibliography, and the publication lists of the authors active in this area) found none.

---

\*email: mark.marosi@egroup.hu. **[AUTHORSHIP/AFFILIATION TO BE CONFIRMED BY THE AUTHOR BEFORE SUBMISSION.]** The search, verification, and manuscript preparation were carried out autonomously by Claude (Anthropic) on a single NVIDIA GH200 node; see Section 3.

The reason the entries treated here were improvable is visible in Kéri’s own source annotations. For moderate  $q$  and  $R \geq 3$  nearly all tabulated upper bounds carry keys denoting *general constructions*: the direct sum

$$K_q(n_1 + n_2, R_1 + R_2) \leq K_q(n_1, R_1) \cdot K_q(n_2, R_2), \quad (1)$$

the special case  $K_q(n + 1, R) \leq_q K_q(n, R)$ , the monotonicity  $K_q(n + 1, R + 1) \leq K_q(n, R)$ , and product rules over alphabet factorizations. These are inherited arithmetic: no search was ever run in those cells. The present note shows that plain local search, seeded with the construction that produced the incumbent, beats all eight such bounds that we attacked in the range  $q \in \{6, 7\}$ ,  $n \leq 10$  — in one case by 22%.

## 2 Results

| cell         | previous bounds | new UB      | $\Delta$ | code file         | previous UB from            |
|--------------|-----------------|-------------|----------|-------------------|-----------------------------|
| $K_6(7, 3)$  | 70–246          | <b>235</b>  | −11      | K6_7_3_M235.txt   | $6 \cdot K_6(6, 3)$         |
| $K_6(8, 3)$  | 246–1080        | <b>1054</b> | −26      | K6_8_3_M1054.txt  | $K_6(4, 1) \cdot K_6(4, 2)$ |
| $K_6(8, 4)$  | 46–216          | <b>169</b>  | −47      | K6_8_4_M169.txt   | $6 \cdot K_6(7, 4)$         |
| $K_6(9, 4)$  | 136–738         | <b>719</b>  | −19      | K6_9_4_M719.txt   | $K_6(3, 1) \cdot K_6(6, 3)$ |
| $K_6(9, 5)$  | 32–144          | <b>126</b>  | −18      | K6_9_5_M126.txt   | alphabet product rule       |
| $K_6(10, 4)$ | 417–2952        | <b>2951</b> | −1       | K6_10_4_M2951.txt | $K_6(4, 1) \cdot K_6(6, 3)$ |
| $K_6(10, 5)$ | 83–615          | <b>610</b>  | −5       | K6_10_5_M610.txt  | $K_6(6, 3) \cdot K_6(4, 2)$ |
| $K_7(8, 4)$  | 76–343          | <b>333</b>  | −10      | K7_8_4_M333.txt   | $K_7(7, 3)$ (monotonicity)  |

Table 1: New upper bounds. “Previous bounds” are the lower–upper bound pairs of Kéri’s tables [2]; lower bounds are unaffected by this work. The last column gives the construction from which the previous upper bound was inherited, per the source keys of [2].

The eight codes are supplied as ancillary files (one codeword per line, digits over  $\{0, \dots, q - 1\}$ ). None of the searches had converged when stopped, so we expect several of these bounds to be improvable further by the same method with more computation.

**Remark 1.** *An audit of the tables of [2] against their own derivation rules found one internal gap: the tabulated  $K_{17}(7, 2) \leq 252735$  is weaker than  $K_{17}(7, 2) \leq 17 \cdot K_{17}(6, 2) = 17 \cdot 14424 = 245208$ , which follows from the tabulated  $K_{17}(6, 2) \leq 14424$  and the standard rule  $K_q(n+1, R) \leq_q K_q(n, R)$ . We record this as an observation; it involves no new construction.*

## 3 Method

### 3.1 Search

We search for a code of prescribed size  $M$  minimizing the number of uncovered words. The state is a multiset of  $M$  codewords together with the coverage count  $\text{cnt}(w)$  of every  $w \in \mathbb{Z}_q^n$ ; a move changes one coordinate of one codeword. The move update is cheap because of the following observation.

**Lemma 1.** *Let  $c'$  be obtained from  $c$  by setting coordinate  $p$  to  $v \neq c_p$ . Then the set of words leaving the radius- $R$  ball of  $c$  and the set of words entering the ball of  $c'$  are both indexed by the words at distance exactly  $R$  from  $c$  in the coordinates other than  $p$ ; consequently a move updates exactly  $2S$  coverage counters with  $S = \binom{n-1}{R}(q-1)^R$ , the two sets are disjoint, and distinct counter updates address distinct words.*

*Proof.* Since  $d(w, c') = d(w, c) + [w_p = c_p] - [w_p = v]$ , a word can leave the ball of  $c$  only if  $d(w, c) = R$  and  $w_p = c_p$ , and can enter the ball of  $c'$  only if  $d(w, c') = R$  and  $w_p = v$ ; writing such a word as its restriction to the coordinates  $\neq p$  (at distance exactly  $R$  from the restriction of  $c$ ) plus its value at  $p$  gives the stated bijections, disjointness ( $w_p = c_p$  versus  $w_p = v$ ), and injectivity.  $\square$

The search itself is a focused local search in the WalkSAT lineage, close in spirit to the tabu searches that produced much of the original tables [6, 5]: pick a random uncovered word  $u$ ; collect as candidate moves every codeword at distance exactly  $R + 1$  from  $u$  nudged one coordinate towards  $u$  (if no codeword is that close, the codewords at minimum distance from  $u$  are used); evaluate all candidates exactly and in parallel; commit a best candidate, ties broken uniformly at random. On stagnation a codeword is teleported onto an uncovered word. Parameter studies on  $K_6(6, 3)$  showed that *candidate breadth dominates*: evaluating all candidate moves exactly outperformed sampling 24 or 64 of them (final uncovered counts 0 versus 82 and 18), while tabu tenures and WalkSAT-style noise were harmful on these instances. Simulated annealing sustained a  $33\times$  higher move rate and still lost to the focused search — move quality beats move count here.

### 3.2 Seeding and descent

For each cell we first rebuilt a code of the incumbent size from the same construction that produced the incumbent bound (e.g., for  $K_6(10, 4)$  the direct sum (1) of an optimal  $K_6(4, 1)$  code of size 72 and a  $K_6(6, 3)$  code of size 41, both found by direct search and verified). Descent then proceeds by remove-and-repair: delete the codeword whose private coverage (words covered by no other codeword) is smallest, and repair the deficit by local search; on success, repeat at  $M - 1$ . Searching the small factors and re-composing (a “factor attack”) was decisive for seeding; the improvements themselves come from the descent breaking out of the product structure.

### 3.3 Hardware and effort

All record searches ran on the 64-core Grace CPU of a single NVIDIA GH200 node (a CUDA port was implemented and verified but was latency-bound and slower than the CPU at these instance sizes). Per-cell effort ranged from seconds ( $K_6(8, 4)$ ,  $K_6(9, 5)$ ) to under two hours of wall clock; none of the descents had converged when stopped. Move throughput ranged from  $\sim 1.3\text{k}$  moves/s per worker on the largest cells to  $\sim 400\text{k}$  on the smallest.

## 4 Verification

False records are worse than no records, so verification is exhaustive, exact, and redundant. Every claimed code was checked by four methods, two of which share no code with the other two:

1. ball marking: mark all words within distance  $R$  of each codeword in a  $q^n$  bitmap and confirm the bitmap is full;
2. meet-in-the-middle bitset covering over a coordinate split;
3. a min-distance scan: enumerate all  $q^n$  words and compute each word's distance to the nearest codeword (this also reports the exact covering radius);
4. Hamming-graph dilation:  $R$ -fold neighborhood dilation of the code's indicator vector, computing no distances at all, with cost independent of  $M$ .

In addition, a fifth, independently written dilation verifier (produced without access to the other implementations) re-confirmed all eight codes from the published files. The largest check ( $q^n = 6^{10} = 60,466,176$  words) completes in seconds.

The ancillary files include `verify_cov.py`, a standalone dependency-free Python verifier: `python3 verify_cov.py CODEFILE q n R` exhaustively re-checks any of the claimed bounds in at most a few minutes.

## 5 Discussion

The pattern in our experiments is clean: local search consistently beat bounds that were inherited from general constructions (relative slack between the previous upper bound and the sphere-covering lower bound of  $4\times-10\times$ ), and consistently failed to improve cells that had already been the object of dedicated search or algebraic construction (slack  $\leq 3\times$ ), such as  $K_3(11, 4) \leq 81$  or  $K_6(6, 3) \leq 41$ . Kéri's tables contain many further cells of the first kind, for  $q \in \{6, \dots, 21\}$  and  $R \geq 3$ ; we expect them to fall to the same method, and a systematic sweep is in progress. Since the tables have had no maintainer since 2011, we also intend to publish a merged, machine-readable bounds table (Kéri 2011 plus the post-2011 lower-bound literature plus the present improvements) alongside the code.

### Data availability

The eight codes, the verifier, and a README are ancillary files of this arXiv submission. The full search and verification source code is available at **[REPOSITORY URL TO BE ADDED]**.

### Acknowledgments

The search software, the verification protocol, and this note were produced autonomously by Claude (Anthropic), running on a dedicated NVIDIA GH200 node. **[TO BE REVIEWED BY THE AUTHOR.]**

## References

- [1] G. Cohen, I. Honkala, S. Litsyn, A. Lobstein, *Covering Codes*, North-Holland, Amsterdam, 1997.
- [2] G. Kéri, Tables for bounds on covering codes, <https://old.sztaki.hu/~keri/codes/> (last updated November 2011).

- [3] D. Gijswijt, S. Polak, Semidefinite lower bounds for covering codes, arXiv:2504.01932, 2025.
- [4] Y. Wu, C. Chen, Improved lower bounds on the domination number of hypercubes and binary codes with covering radius one, Discrete Mathematics 347 (2024) 113752.
- [5] P. J. M. van Laarhoven, E. H. L. Aarts, J. H. van Lint, L. T. Wille, New upper bounds for the football pool problem for 6, 7 and 8 matches, J. Combin. Theory Ser. A 52 (1989) 304–312.
- [6] P. R. J. Östergård, Constructing covering codes by tabu search, J. Combin. Des. 5 (1997) 71–80.
- [7] G. Kéri, P. R. J. Östergård, Further results on the covering radius of small codes, Discrete Mathematics 307 (2007) 69–77.
- [8] A. Florath, A Lean 4 formalization of  $q$ -ary covering codes with a proof-carrying database of certified bounds, arXiv:2606.09600, 2026.

## A The code proving $K_6(8, 4) \leq 169$

For self-containedness we print the largest single improvement ( $216 \rightarrow 169$ ,  $-22\%$ ). Each row lists codewords of  $\mathbb{Z}_6^8$ ; the machine-readable version is the ancillary file `K6_8_4_M169.txt`.

```

13413135  53400455  13125414  32253533
12551004  52454202  05521440  33342502
44555514  41031103  53240312  04212401
00435123  02202350  52021112  12504245
11000212  02305532  25241223  23003413
10223022  50525033  34305303  43010554
20021234  35112313  01520005  55010150
34524104  40344155  24515055  50215100
01144002  33011251  53435304  13555222
24052113  50114555  40504340  14444315
54353423  10330254  22205053  22532152
24311332  21432553  43352224  45225545
42315325  11354522  31303134  21100125
45114110  54155011  10142111  20501500
42231421  23422142  42123454  14350320
31135242  34133540  40013302  35223200
30052351  31541413  50532514  43535310
30004423  35445522  24420251  00301511
21105430  32210044  51513442  51320111
05334013  30550145  51412420  21123311
21204204  20453540  53333231  00431214
04024022  20345400  32211102  02541334
43041530  04203451  50050043  01451451
02412541  32433415  01033525  22343044
40420300  52034330  44101542  03023020
03153105  41443143  22415214  41224331
14035140  25014324  15303352  35424050
55553210  54243154  03122335  25320502
34321545  10402034  35054444  12122225
05402120  41252412  53444041  01250344
13214510  23550000  44332024  34314011
44530022  03312243  22524521  45542205
14042405  42411413  12040511  01351250
44504231  15545434  32100114  13322503
43401052  31530353  31220030  03154453
05045554  04240553  55101324  12055451

```

|          |          |          |          |
|----------|----------|----------|----------|
| 23234445 | 11512050 | 45250501 | 31425531 |
| 20131035 | 55002001 | 15131341 | 33501125 |
| 10421303 | 45450133 | 25052035 | 23505141 |
| 55351335 | 44205215 | 51045345 | 52130203 |
| 15330401 | 15113023 | 34140230 |          |
| 50222243 | 00110432 | 00543221 |          |
| 05234132 | 11211533 | 22142150 |          |